



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

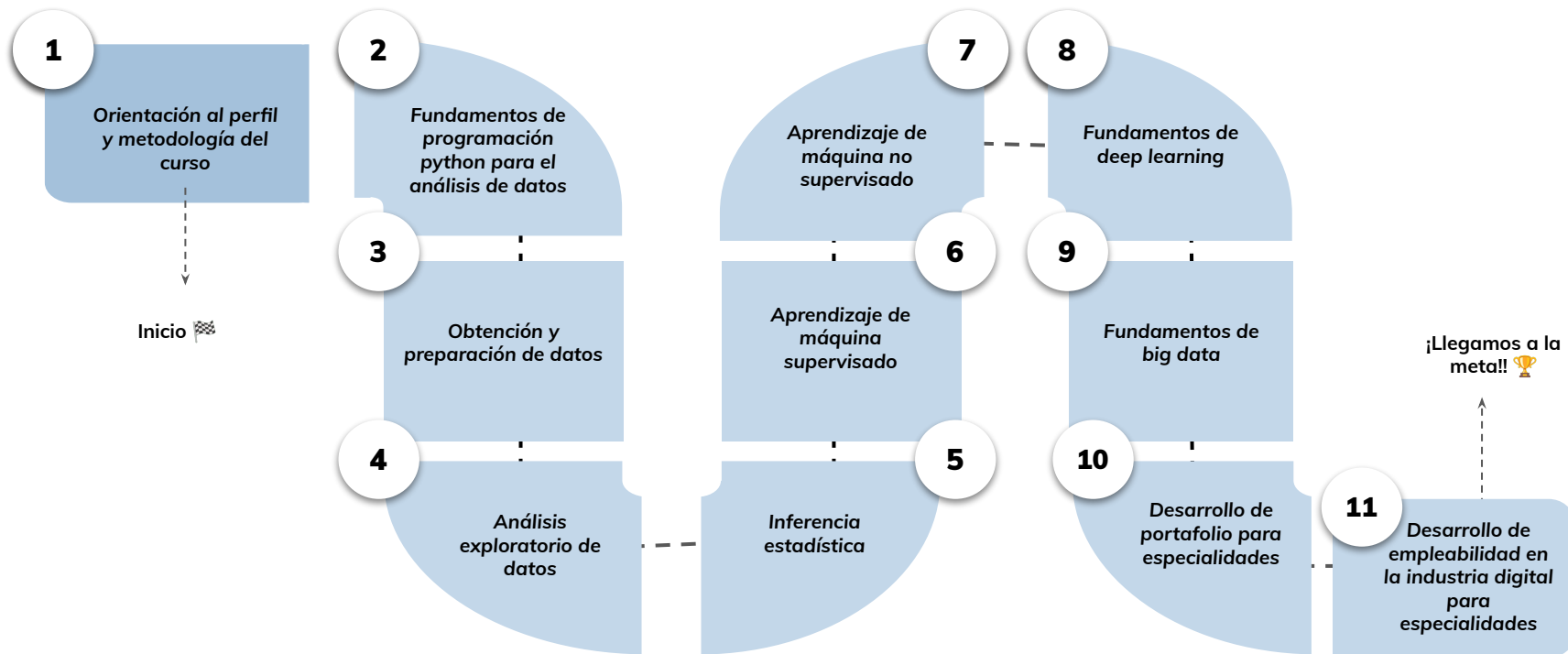


› Implementación de rn en python- Parte I

Aprendizaje Esperado 3: Implementar una red neuronal artificial utilizando python para resolver un problema de regresión y/o clasificación.

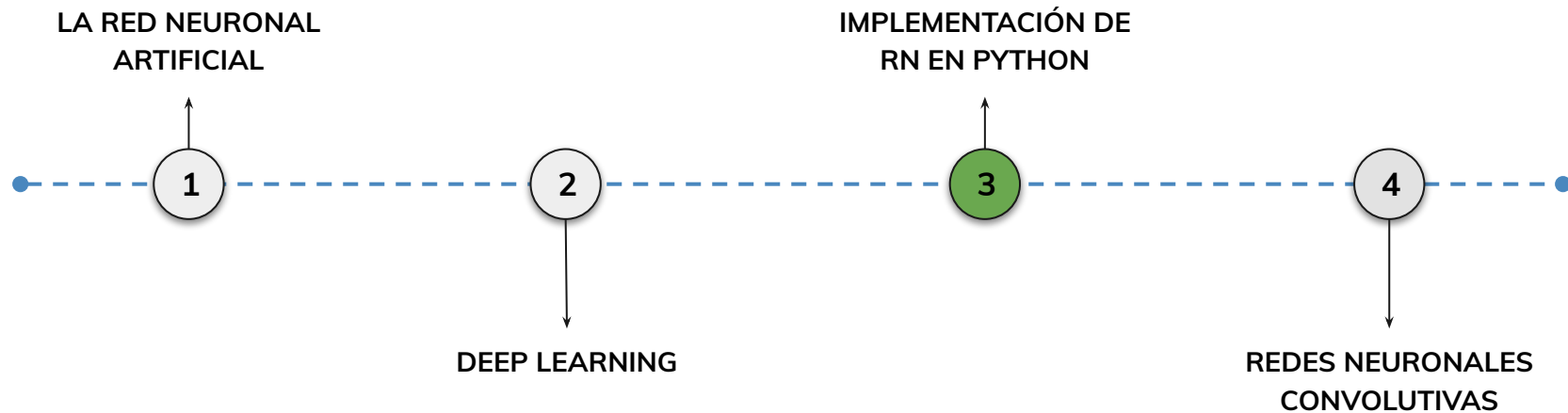
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

1.

Implementación de redes neuronales en Python

Profundizaremos en los conceptos y componentes fundamentales para implementar redes neuronales utilizando Python y librerías de Deep Learning.

Entorno y bibliotecas para
redes neuronales

Python como
lenguaje principal
para IA.

Librerías clave:
Keras y PyTorch.

Diseño conceptual de
redes neuronales

Tensores y su
manipulación.

Arquitecturas para
regresión y
clasificación.



Objetivos de aprendizaje

¿Qué aprenderás?



- Reconocer los principales entornos y bibliotecas de desarrollo en Python para redes neuronales.
- Comprender el concepto de tensores y su importancia en el flujo de datos.
- Diferenciar arquitecturas de redes neuronales para regresión y clasificación.
- Conocer las funciones de activación, funciones de pérdida y optimizadores más comunes.
- Identificar técnicas para evaluar modelos y evitar el sobreajuste.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Reforzamos el proceso de entrenamiento en Deep Learning (preparación de datos, diseño, entrenamiento, evaluación y despliegue).
- Identificamos los principales desafíos del DL: datos, recursos computacionales, interpretabilidad y sobreajuste.
- Exploramos técnicas avanzadas: regularización, optimización de hiperparámetros y Transfer Learning.
- Conocimos redes generativas (GANs), modelos de difusión y aprendizaje por refuerzo profundo.

Implementación de rn en python

Implementación Práctica de Redes Neuronales en Python

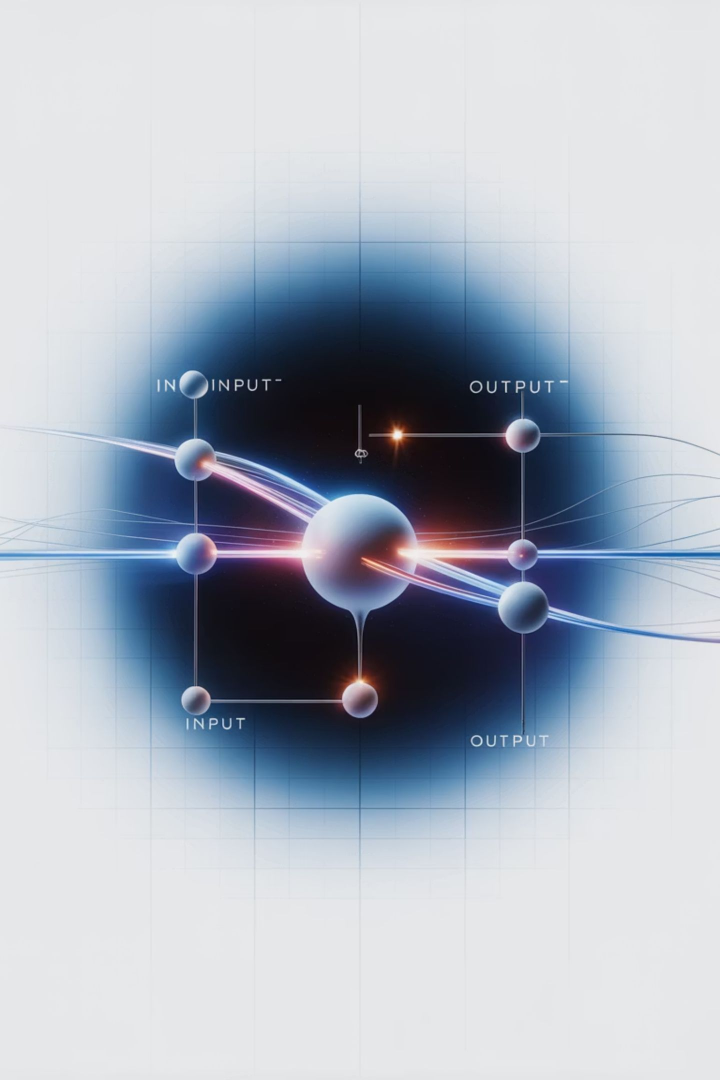
El desarrollo de redes neuronales artificiales ha sido potenciado significativamente gracias a la disponibilidad de herramientas de programación accesibles y poderosas como Python. Este lenguaje, junto con librerías como Keras y PyTorch, ha permitido a estudiantes, investigadores y profesionales crear modelos de aprendizaje automático sin necesidad de una infraestructura compleja o conocimientos avanzados de matemáticas aplicadas.

Objetivo del Manual

Este manual tiene como objetivo guiarte paso a paso en la **implementación práctica de una red neuronal** orientada a resolver tanto problemas de regresión como de clasificación.

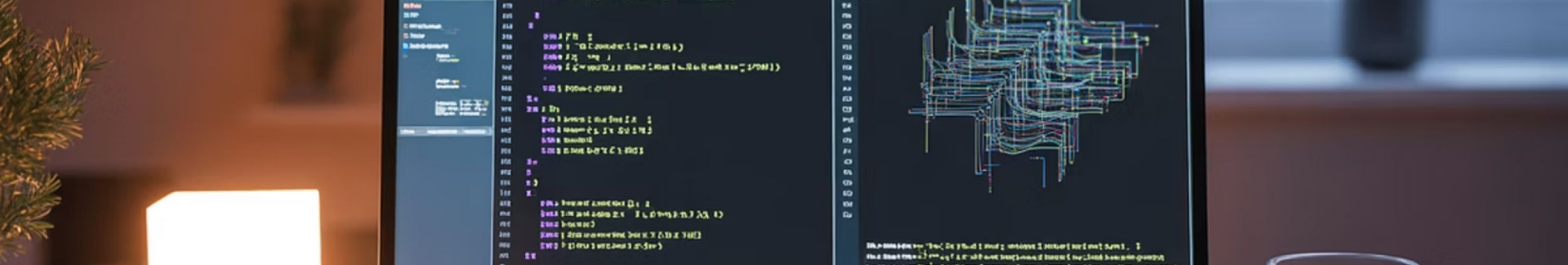
A lo largo de este contenido teórico, aprenderás sobre los conceptos clave que forman parte del entrenamiento y la evaluación de redes neuronales, como tensores, funciones de activación, optimizadores, regularización, overfitting y más.





Importancia de los Conceptos Fundamentales

La comprensión de estos elementos es fundamental para diseñar modelos eficientes, generalizables y efectivos en distintos tipos de datos.



El Entorno Python para Redes Neuronales

Python se ha consolidado como el lenguaje líder en inteligencia artificial debido a su sintaxis simple, gran comunidad, y la amplia variedad de librerías especializadas.

Principales Bibliotecas para Redes Neuronales



Keras

Ofrece una interfaz de alto nivel que permite crear y entrenar redes neuronales con pocas líneas de código. Se integra directamente con TensorFlow.



PyTorch

Permite mayor control del modelo gracias a su diseño dinámico. Es ampliamente usado en investigación.

Ambos entornos permiten trabajar con GPU, realizar experimentos reproducibles, visualizar resultados y escalar modelos a producción.

Comparativa: Keras vs PyTorch

Característica	Keras	PyTorch
Facilidad de uso	Muy alto	Alta
Curva de aprendizaje	Baja	Media
Popularidad en investigación	Media	Alta
Documentación	Muy completa	Completa
Integración con TensorFlow	Nativa	Parcial

Elección de la Biblioteca Adecuada

La elección entre Keras y PyTorch dependerá de los objetivos del usuario:

- Keras es ideal para comenzar
- PyTorch es recomendable para proyectos más complejos o experimentales



¿Qué es un Tensor?

Un **tensor** es la unidad básica de datos en los frameworks de Deep Learning. Se puede entender como una generalización de matrices.

Tipos de Tensores

Escalar

Tensor de 0 dimensiones (número).

Matriz

Tensor de 2D.

Vector

Tensor de 1D.

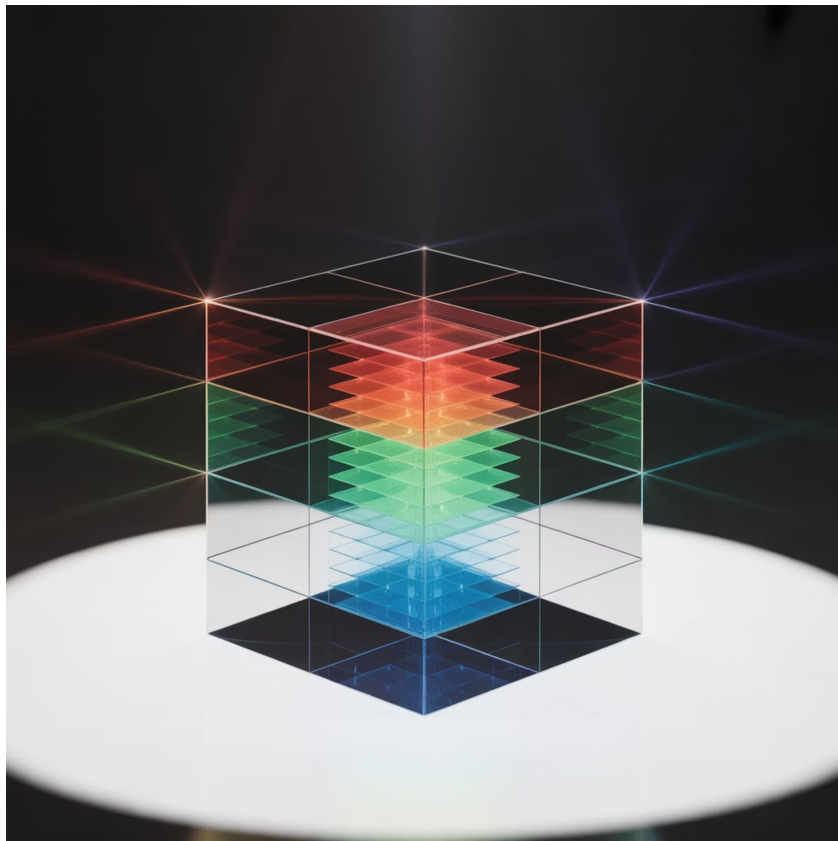
Tensor n-dimensional

Datos complejos como imágenes, secuencias o lotes de datos.

Ejemplo de Tensor en Imágenes

Por ejemplo, una imagen en color de 64x64 píxeles con 3 canales (RGB) se representa como un tensor de dimensiones (64, 64, 3).

Durante el entrenamiento, los tensores fluyen a través de las capas de la red, sufriendo transformaciones lineales y no lineales hasta producir una predicción.



Manipulación de Tensores

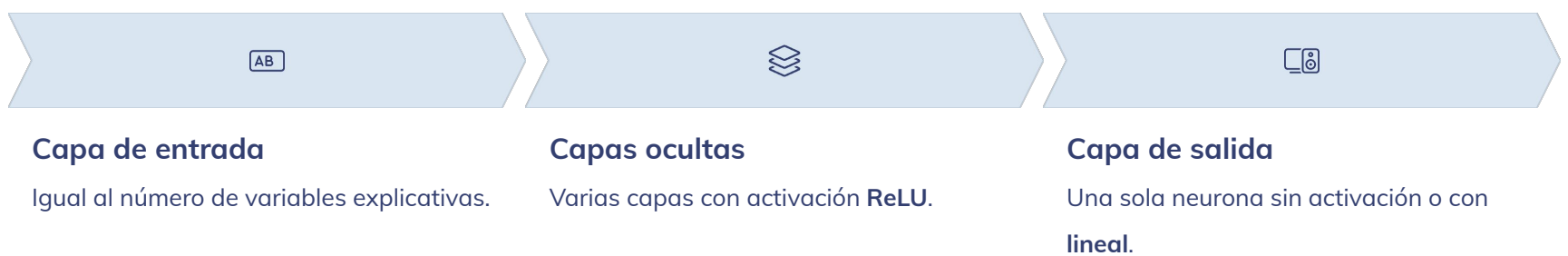
En Keras o PyTorch, los tensores se crean automáticamente al cargar datos y son manipulados mediante operaciones matemáticas optimizadas para CPU y GPU.



Diseño de una Red Neuronal para Regresión

Una red para regresión predice un valor numérico continuo.

Estructura Típica de una Red para Regresión



AB

Capa de entrada

Igual al número de variables explicativas.



Capas ocultas

Varias capas con activación **ReLU**.



Capa de salida

Una sola neurona sin activación o con **lineal**.

Componentes de Evaluación para Regresión

Función de pérdida

Mean squared error (MSE)

Métricas

- Mean absolute error (MAE)
- R^2
- Otras métricas específicas



Ejemplo Conceptual en Keras para Regresión

Este modelo será entrenado sobre datos reales y evaluado en conjunto de validación. Ajustes como el número de capas y neuronas dependerán de la complejidad del problema.

Diseño de una Red Neuronal para Clasificación

En clasificación, el objetivo es asignar una etiqueta a una entrada.

Tipos de Clasificación

Clasificación Binaria

Una neurona con activación **sigmoid**.

Ejemplo: Detección de spam (spam/no spam)

Clasificación Multiclase

n neuronas con activación **softmax**.

Ejemplo: Reconocimiento de dígitos (0-9)



Cross-Entropy Loss

Funciones de Pérdida para Clasificación

La función de pérdida más usada es `categorical_crossentropy` o `binary_crossentropy`.

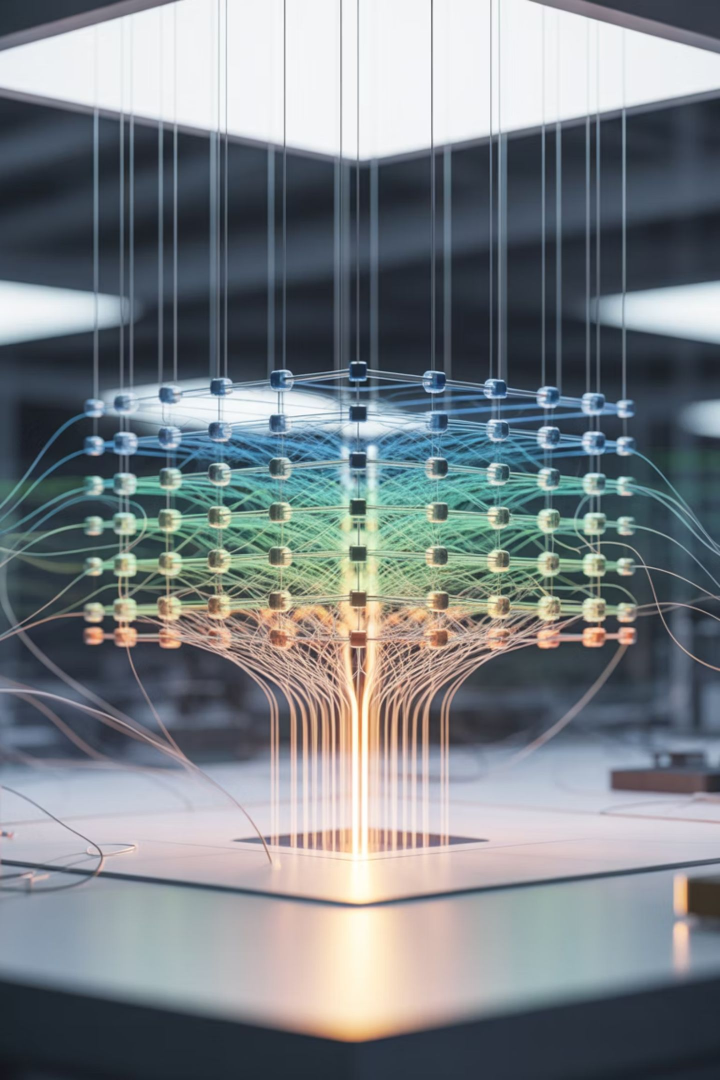
Ejemplo Conceptual en PyTorch

Este modelo es adecuado para tareas como clasificación de imágenes, textos o señales.

```
import torch.nn as nn

class RedClasificacion(nn.Module):
    def __init__(self):
        super(RedClasificacion, self).__init__()
        self.layer1 = nn.Linear(n_features, 64)
        self.layer2 = nn.Linear(64, 32)
        self.output = nn.Linear(32, n_classes)

    def forward(self, x):
        x = torch.relu(self.layer1(x))
        x = torch.relu(self.layer2(x))
        return torch.softmax(self.output(x), dim=1)
```



Funciones de Activación

Las funciones de activación permiten introducir no linealidad en la red.

Funciones de Activación Comunes

Función	Fórmula / Notación	Uso habitual
ReLU	$\max(0, x)$	Capas ocultas, rápido y estable
Sigmoid	$1 / (1 + e^{-x})$	Clasificación binaria (salida)
Softmax	$e^{x_i} / \sum e^{x_j}$	Clasificación multiclase
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	Menos usada, reemplazada por ReLU

Elección de Funciones de Activación

La elección depende de la tarea y la capa. ReLU es la más eficiente en capas intermedias por su simplicidad computacional.

Función de Pérdida

Evalúa qué tan bien está funcionando la red:

Regresión

- `mean_squared_error`
- `mean_absolute_error`

Clasificación

- `binary_crossentropy`
- `categorical_crossentropy`

Optimizadores

Se encargan de ajustar los pesos de la red:

SGD

Descenso de gradiente estocástico



Adam

Adaptativo, recomendado en la mayoría de los casos



Learning Rate

Define cuánto se ajustan los pesos por iteración.

- Si es demasiado alto, puede no converger
- Si es muy bajo, el entrenamiento será lento



Epochs

Cada epoch representa una pasada completa por los datos de entrenamiento.

Se debe ajustar según el comportamiento de la pérdida y la validación.





Evaluación y Optimización de la Red

La evaluación se realiza comparando el rendimiento en un conjunto de validación con el conjunto de entrenamiento.

Métricas Comunes de Evaluación



Accuracy

Porcentaje de predicciones correctas.



MAE/MSE

Errores medios para problemas de regresión.



Precision / Recall / F1-score

Métricas para evaluar clasificadores.

Técnicas de Evaluación

Validación cruzada

Divide los datos en múltiples conjuntos para entrenar y validar.

Early Stopping

Detener el entrenamiento si no mejora.

Ajuste de hiperparámetros

Número de capas, neuronas, learning rate, etc.



Validación y Regularización

El problema del sobreajuste en las redes neuronales

¿Qué es el Sobreajuste?

El sobreajuste ocurre cuando la red memoriza los datos de entrenamiento y no generaliza a datos nuevos.



**Overfitting
Model**

**Generalization
Model**

Técnicas para Mitigar el Sobreajuste

19

Dropout

Desactiva aleatoriamente neuronas durante el entrenamiento.



L2 regularization

Penaliza pesos altos.



Aumento de datos

En problemas de imagen o texto.



Reducir la complejidad del modelo

Usar menos capas o neuronas.

Live Coding

¿En qué consistirá la Demo?

Mostraremos los pasos necesarios para implementar una red neuronal artificial en Python capaz de resolver un problema de regresión o clasificación.

1. Preparar los datos de entrada (dataset para regresión y clasificación).
2. Configurar el entorno de trabajo en Python (instalación de Keras o PyTorch).
3. Construir conceptualmente la arquitectura de una red neuronal:
 - a. Capas de entrada, ocultas y de salida.
 - b. Selección de funciones de activación (ReLU, Sigmoid, Softmax).
4. Definir la función de pérdida y el optimizador (MSE, Crossentropy, Adam).
5. Configurar hiperparámetros: número de epochs, tamaño de batch y learning rate.
6. Entrenar la red con datos de ejemplo y monitorear las métricas.
7. Evaluar el rendimiento y explicar cómo ajustar los parámetros si fuera necesario.



Momento:

Time-out!



5 -10 min.





Ejercicio N° 1

Implementa tu propia red neuronal artificial

Implementa tu propia red neuronal artificial

Contexto: 🙌

Vas a poner en práctica los conceptos aprendidos creando y entrenando tu primera red neuronal artificial en Python.

Consigna: 🛠️

1. Selecciona un dataset sencillo (puede ser de regresión o clasificación).
2. Diseña la arquitectura de la red neuronal:
 - a. Define cuántas capas ocultas y neuronas tendrá cada una.
 - b. Escoge las funciones de activación adecuadas.
3. Define la función de pérdida, el optimizador y las métricas de evaluación.
4. Entrena el modelo y documenta el proceso:
 - a. ¿Cómo evolucionaron las métricas en cada epoch?
 - b. ¿Cómo evitaste el sobreajuste?
5. Presenta los resultados finales y analiza si la red cumple con el objetivo del dataset elegido.

Paso a paso: ⚙️

1. Prepara y divide los datos en entrenamiento y validación.
2. Crea y configura el modelo en Keras o PyTorch.
3. Ajusta hiperparámetros hasta obtener buenos resultados.
4. Presenta el rendimiento final con métricas claras (MAE, Accuracy, F1, etc.).

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Aprendimos a implementar una red neuronal artificial en Python.**
- ✓ **Diseñamos redes para resolver problemas de regresión y clasificación.**
- ✓ **Conocimos los pasos para preparar datos, definir arquitecturas, entrenar y evaluar un modelo.**
- ✓ **Identificamos los hiperparámetros clave para optimizar el desempeño.**
- ✓ **Vimos cómo interpretar resultados y ajustar el modelo según las métricas obtenidas.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

